PayFlow Events

Lossless, skew-resilient payment event pipeline. Goal: prove 0 lost, 0 phantom and 0 duplicate side effects under crash injection with a transactional outbox (and count what naive dual-write loses), keep p99 flat under 40% merchant skew, and keep healthy events flowing past poison pills. Full design and plan: `docs/plan.md`. Outbox contract: `docs/contract/outbox.sql` (owned by PayFlow Ledger). Progress: `docs/progress.md`.

Stack (pinned, do not upgrade mid-phase)

- Java 25, virtual threads. Spring Boot 4.0.x, Spring for Apache Kafka 4.x. Jackson 3 (`tools.jackson.*`).
- Apache Kafka 4.1.x, KRaft only, 3 brokers, RF=3, `min.insync.replicas=2`, `auto.create.topics.enable=false`.
- Debezium 3.4+ Postgres connector (`pgoutput`) + Outbox Event Router on Kafka Connect.
- PostgreSQL 18 (`wal_level=logical`). Apicurio Registry 3 with JSON Schema, BACKWARD compatibility.
- Kafka Streams with `exactly_once_v2` and the classic rebalance protocol.
- Telemetry: spring-boot-starter-opentelemetry, Kafka observation, opentelemetry-logback-appender, JMX exporter agents on brokers and Connect, OTel Collector (otlp, kafkametrics, prometheus, postgresql, sqlquery, docker_stats) to Prometheus 3, Tempo, Loki, Grafana.
- Tests: JUnit 5, Testcontainers (Kafka, Postgres, Debezium), Awaitility, Toxiproxy.

Modules

`common` (record shape, headers, schemas, `EventsMetrics`, salting) · `source-app` · `dual-write-app` (experiment only) · `relay-poller` · `consumer` (+ DLQ replay CLI) · `rollup-streams` · `verifier` · `generator` · `connect/` · `chaos/` · `perf/` · `deploy/`

Commands

```
make up            # brokers, registry, postgres, connect, apps, telemetry stack
make down          # stop and remove volumes
make test          # unit + schema compatibility + contract tests
make it            # Testcontainers integration: idempotency, ordering, CDC
make chaos         # chaos matrix, verifier per run; non-zero exit on any outbox FAIL
(MATRIX=smoke for a short run)
make skew          # skew lab sweep, resumable (MATRIX=smoke for 2 runs)
make pills         # poison-pill runs with and without pills
make rebalance     # rolling-restart benchmark for 3 group protocols
make report        # Prometheus API + results.json -> docs/results.md and PNGs
```

Run `make test it` before every commit. Grafana :3000, Kafka UI :8085, registry :8081, Connect :8083.

Delivery rules (never break, never weaken to pass a test)

1. An event exists only if its outbox row committed. Apps never call `producer.send()` for business events; only relays do. `dual-write-app` is the single exception and exists only to be measured.
2. Both relays produce the same record: key = `aggregate_id`, value = outbox `payload` as-is, headers `id, type, seq, occurred_at, traceparent, merchant_id`. The contract test guards it.
3. Producers: `enable.idempotence=true`, `acks=all`. Never lower these.
4. Consumers: auto-commit off. Dedup insert, side effect and seq update in one DB transaction. Commit Kafka offsets only after the DB commit.
5. Dedup key is the `id` header (uuidv7). `ON CONFLICT DO NOTHING`; apply effects only for inserted ids.
6. Partition key is the entity (`aggregate_id`), never `merchant_id`. Merchant totals come from the two-stage salted rollup.
7. Failures go to retry topics then the DLQ; never retry in-place on the main partition. Non-retryable errors (deserialization, validation) go straight to the DLQ.
8. Topics are created only by the topic-init script with the configs in plan section 3 (`LogAppendTime` on event topics). `@RetryableTopic` never auto-creates.
9. The outbox schema is the Ledger's contract. Never change it here; `V1` must equal `docs/contract/outbox.sql`.
10. Never edit an applied Flyway migration. Add a new one.

If a task seems to need breaking a rule, stop and say so instead.

Telemetry conventions

- Every custom meter is registered in `common/.../EventsMetrics`. Names and labels exactly as plan section 5.
- OTel names use dots (`payflow.events.e2e.latency`); Prometheus shows `payflow_events_e2e_latency_seconds`.
- Latency split: relay = record timestamp (broker LogAppendTime) minus `occurred_at`; consume = now after DB commit minus record timestamp; e2e = their sum. Histograms with the SLO buckets in plan section 4.
- Allowed labels: `topic, partition, relay, result, kind, stage, poison, strategy, group, scenario, run_id`. Never label by event, payment, aggregate or merchant id.
- Lag is reported in seconds (`payflow_events_consumer_lag_seconds`), not only messages.
- Trace context: relays copy `traceparent` into the Kafka header; consumers continue it via Spring Kafka observation. Never start a new root trace in the consumer when the header exists.
- Every script sets `RUN_ID`, `SCENARIO`, `RELAY` before starting apps; they become resource attributes and Prometheus labels.
- JMX metric names come from `deploy/observability/jmx/*.yml`. If you rename a rule, update plan section 5 and dashboards in the same commit.
- Dashboards, alerts, recording rules, Collector config and connector JSON are files in the repo. No manual Grafana or Connect REST steps.

Testing rules

- Integration tests use real Kafka and Postgres via Testcontainers. No embedded Kafka for correctness tests.
- The verifier is the single oracle for lost, phantom, duplicate effects, order and rollup mismatch. Tests and chaos runs use it; do not write a second definition of correct.
- Use Awaitility for async assertions; never `Thread.sleep` to wait for delivery.
- A flaky test is a bug: find the race or the missing wait condition.
- Bench and chaos numbers come only from `make report`; every result row cites its `RUN_ID` and machine spec.

Workflow

- One runbook prompt = one commit. Message: `<prompt-id>: <what changed>`.
- Plan first on [plan] prompts: list modules and files to touch, then build.
- After each prompt append one line to `docs/progress.md`: id, change, check result, RUN_IDs.
- Tuning follows the loop: one change behind config, BEFORE/AFTER RUN_IDs, keep only if correctness is 0 and gain >= 5%.
- Long sweeps (chaos, skew) are resumable via state files; never restart a sweep from scratch to fix one run.
- When something fails, state the root cause in one sentence before editing code.

Don’t

- Don’t use `KafkaTemplate` in business services; only relays and the DLQ replay CLI produce.
- Don’t catch and swallow exceptions in listeners; throw so the retry/DLQ path sees them.
- Don’t raise partition counts on an existing topic in tests; recreate the topic (keys remap).
- Don’t load full event-id sets into memory in the verifier when a SQL anti-join works.
- Don’t commit ad-hoc Grafana snapshots or results; only `make report` output for runs listed in progress.md.